third-party library that turns your Django project into a static site. You install Distill using the familiar `python -m pip install django-distill` and add `"django_distill"` to `INSTALLED_APPS`.

Distill works by replacing the `path()` function in your urls.py files with its own: `distill_path()`. This function overrides `path()`, so your site continues to work as it did before. The new version of the function also allows you to run a management command that outputs a static version of all the URLs mapped by Distill. The `distill_path()` function takes at least one more argument than the regular kind: the name of the file to generate for the view.

As an example, I maintain an online cookbook for my family, containing some of our favorite recipes. Since the site is just for the family to use, and I'm the only one to ever edit it, using a content management system seemed like overkill. The original version of the site was built on a third-party Django project that required Django 2.1. As the library never got updated, maintaining the site became problematic. Instead of rewriting someone else's project for my own use, I decided to downshift to a static site. The recipes are in data files, and a simple view renders them as HTML. Instead of hosting a Django site, I use Distill to create a static site with all the recipes. As it's a static site, hosting it is as simple as zipping it up and sending it to a server.

There are a few restrictions to what Distill can do. First off, it only works with the GET mode of views. This makes sense, as doing a POST is part of a dynamic site; there won't be any Django on the backend to handle a POST with statically generated files. Second, if your view takes arguments, you need to specify which of those arguments to use when generating the static site. Distill allows you to pass in a function to generate the arguments, meaning you can dynamically build queries at generation time. In fact, I use this in my cookbook: in that case, the function examines the data directory for each recipe file and calls the view with the recipe name. The result is that Distill generates an HTML static page for each recipe.

Appendix B provides an overview of how to put your Django project into a production environment. Part of this process includes taking the static files in your project and deploying them to a web server, letting it serve them. It is inefficient to have Django serve static files, and you should only do it that way during local development. There is a management command called `collectstatic`, which is responsible for copying your static files to the web server's required destination. Distill expects this to be set up and working; it uses the same destination to source any static files for the generated site.

Once you have your URLs configured and your static files properly organized, you run the `distill-local` management command. This takes one argument: the output directory for your generated site. Once your site gets generated, you can zip up the output directory and deploy it wherever you plan on hosting your static site.

I really like Distill. Being familiar with Django already, it requires very little extra effort to build a static site. I already know Django's template language; I know how views work; I can test the site using the development server; and, most importantly, I don't have to learn some other static site generating tool.